A

MAJOR PROJECT REPORT

ON

"*RAMA AI: A Comprehensive Literature Review on Offline Large Language Model Inference for Android Mobile Devices Using Flutter and llama.cpp*"

Submitted Impartial Fulfilment for the Award of the Degree of

Bachelor of Technology in

Department of Computer Science & Engineering

Bikaner Technical University, Bikaner (2026-27)

April 2026

Submitted To: Mr. Jai Bhasker

Submitted By: Sandeep Muhal

Roll No: - 22EBUCS045

BIKANER TECHNICAL UNIVERSITY

Session 2026-27

CERTIFICATE

This is to certify that the project titled **"*RAMA AI: A Comprehensive Literature Review on Offline Large Language Model Inference for Android Mobile Devices Using Flutter and llama.cpp*"** is a bona fide record of the work carried out by **Sandeep Muhal** under my supervision and guidance, in partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology**.

Supervisor:

Mr. Jai Bhaskar

Assistant Professor

Candidate's Declaration

I hereby declare that the work, which is being presented in the report entitled "***RAMA AI: A Comprehensive Literature Review on Offline Large Language Model Inference for Android Mobile Devices Using Flutter and llama.cpp***" in partial fulfilment for the award of Degree "**Bachelor of Technology**" in Department of Computer Science and Engineering, and submitted to the **Department of Computer Science & Engineering**, Bikaner Technical University is record of my own investigations. I have not submitted the matter presented in this report anywhere for the award of any other degree.

Sandeep Muhal

22EBUCS045

Department of Computer Science & Engineering BTU, Bikaner

Counter Signed by

Mr. jai Bhaskar

Assistant Professor

Department of Computer Science & Engineering

Acknowledgement

I express my sincere gratitude to **Dr. Jai Bhaskar**, Assistant Professor, Department of Computer Science & Engineering, for his valuable guidance, continuous support, and encouragement throughout the development of this project.

I am also thankful to the faculty members of the Department of Computer Science & Engineering, Bikaner Technical University, for providing the necessary academic environment and resources required for successful completion of this work.

I would like to thank my friends and classmates for their suggestions, motivation, and cooperation during the project.

Finally, I express my heartfelt gratitude to my family for their constant support, trust, and encouragement throughout my academic journey.

Sandeep Muhal

22EBUCS045

ABSTRACT

The rapid growth of Artificial Intelligence has created strong demand for intelligent assistants capable of understanding and generating human language. Most existing AI assistants rely on cloud-based services, which require internet connectivity, involve recurring API costs, and raise privacy concerns. This project presents RAMA AI, a fully offline mobile AI assistant designed to run Large Language Models directly on Android devices.

RAMA AI is developed using Flutter for the user interface, Dart FFI for communication with native libraries, and llama.cpp for efficient on-device model inference. The system uses quantized GGUF language models to reduce memory usage while maintaining useful response quality. All user conversations are stored locally using SQLite, ensuring privacy and offline availability.

The application provides a modern chat interface, conversation history management, model switching capability, and responsive performance through background isolate processing. Since inference is performed locally on the device, no user data is transmitted to external servers.

This project demonstrates that practical AI assistants can be deployed on consumer smartphones without dependence on cloud infrastructure. RAMA AI offers a privacy-focused, cost-effective, and accessible solution for offline conversational AI, especially in environments with limited or unreliable internet connectivity.

Table of Contents

CERTIFICATE II
Candidate Declaration III
Acknowledgement IV
Abstract V

1 Introduction IX
1.1 Background and Motivation IX
1.2 Problem Statement X
1.3 Objectives X
1.4 Scope of Work XI
1.5 Summary XII

2 Literature Review XIII
2.1 Introduction XIV
2.2 Transformer Architectures and Small Language Models XIV
2.3 Model Quantization Techniques and GGUF Ecosystem XV
2.4 Mobile AI Frameworks and Inference Engines XVI
2.5 Flutter and Dart FFI XVI
2.6 Conversational Memory Management XVII
2.7 Privacy-Preserving AI Design XVIII
2.8 Chapter Summary XVIII

3 System Design and Architecture XIX
3.1 Introduction XIX
3.2 Project Overview XIX
3.3 Platform Requirements XXI
3.4 High-Level Architecture XXII
3.5 Layer-Wise Explanation XXIII
3.6 Data Flow Architecture XXV
3.7 Technology Stack XXV
3.8 Why These Technologies Were Chosen XXV
3.9 Performance Design Decisions XXVI
3.10 Security and Privacy Architecture .....XXVI–XXVII
3.11 Scalability of Architecture XXVII
3.12 Chapter Summary XXVII

4 Implementation Details XXVIII
4.1 Native C++ Inference Engine XXVIII
4.2 Android NDK Build Pipeline XXVIII
4.3 Flutter FFI Bridge XXVIII
4.4 Dart Isolate Concurrency Strategy XXIX
4.5 Sliding-Window Context Algorithm XXIX
4.6 Theme System XXIX

5 Inference Pipeline (Deep Dive) XXX
5.1 End-to-End Data Flow XXX
5.2 C++ Inference Stages XXX
5.3 Sampler Chain Configuration XXX

6 Performance and Memory Analysis XXXI
6.1 Introduction XXXI
6.2 Performance Architecture XXXI
6.3 Memory Management Strategy XXXII
6.4 Key Design Trade-offs XXXV
6.5 Real-World Performance Expectations XXXVI
6.6 Reliability-Oriented Engineering Philosophy XXXVII
6.7 Chapter Summary XXXVII

7 Database Design XXXVIII
7.1 SQLite Schema XXXVIII
7.2 Query Patterns XXXVIII

8 Screenshots and Results XXXIX
8.1 Home Screen / Chat Interface XXXIX
8.2 Conversation Interface XL
8.3 Conversation Sidebar XLI
8.4 Model Manager XLII
8.5 Settings Screen XLIII

9 Conclusion and Future Work XLIV
9.1 Conclusion XLIV
9.2 Research Gaps XLIV

9.3 Future Roadmap XLIV
10 REFERENCES XLIV

Chapter 1: Introduction

1.1 Background and Motivation

The rapid advancement of Artificial Intelligence (AI) in recent years has significantly transformed the field of Natural Language Processing (NLP). Among the most influential developments is the emergence of transformer-based Large Language Models (LLMs), which have redefined how machines understand, generate, and interact using human language. Models such as OpenAI GPT series, Meta AI LLaMA family, Google DeepMind Gemma, and Microsoft Phi series have demonstrated impressive capabilities in question answering, summarization, translation, code generation, reasoning assistance, and conversational dialogue.

These systems have become highly effective across personal, educational, and enterprise use cases. Students use them for learning support, developers use them for coding assistance, businesses deploy them for customer service automation, and professionals rely on them for productivity enhancement. Their ability to generate context-aware and human-like responses has made LLMs one of the most impactful innovations in modern computing.

Despite their success, most existing AI assistants depend heavily on cloud-based infrastructure. In this model, user input is transmitted to remote servers where model inference is performed, and responses are then returned over the internet. Although powerful, this approach introduces several limitations. First, cloud inference creates latency caused by network communication, which reduces responsiveness. Second, it introduces recurring operational costs due to API usage, subscriptions, or server hosting. Third, and most importantly, it raises serious concerns regarding privacy and data security, since user conversations may be stored, logged, or analyzed by third-party providers.

To address these limitations, the concept of Edge AI has gained significant importance. Edge AI refers to deploying intelligent systems directly on local devices rather than relying on centralized cloud servers. Smartphones are currently the most widespread edge computing devices in the world, with billions of active users globally. Modern Android devices now include multi-core ARM processors, increasing RAM capacity, efficient storage systems, and advanced power management, making them capable of running optimized AI workloads locally.

Running LLMs directly on smartphones offers multiple advantages. It enables offline access without internet dependency, ensures faster response times by eliminating network delay, reduces long-term service costs, and most importantly preserves user privacy because all computation occurs on the device itself. This creates an opportunity to democratize AI access by making intelligent assistants available to users regardless of network availability or financial constraints.

However, deploying LLMs on mobile devices remains a challenging engineering problem. Most language models require substantial memory, high computational throughput, and optimized runtime environments. Smartphones, in contrast, operate under limited RAM, battery constraints, thermal restrictions, and background process limitations imposed by Android.

This project, **RAMA AI**, is developed as a practical solution to these challenges. It is a fully offline Android AI assistant that performs on-device LLM inference using quantized GGUF models through llama.cpp integration, while providing a modern Flutter-based user interface. The project demonstrates that privacy-preserving conversational AI can be delivered efficiently on consumer smartphones without requiring cloud services.

1.2 Problem Statement

Although conversational AI systems have become widely popular, their current deployment models suffer from several practical and technical limitations. These problems motivate the need for a local, privacy-focused, and efficient mobile AI solution.

Major Problem Identified

1.Privacy and Security Risks in Cloud AI Systems:

Most AI chatbots require user prompts to be transmitted to remote servers. This creates privacy risks because sensitive conversations, personal queries, academic data, and business information may be exposed to third-party systems. It also creates compliance challenges under regulations such as GDPR and CCPA.

2. Internet Dependency and Limited Accessibility

Cloud-based assistants require stable internet connectivity. In areas with weak networks, limited mobile data, or no internet access, such systems become unreliable or unusable.

3. High Operational Costs

Cloud AI platforms often involve recurring API charges or subscription costs. This makes continuous usage expensive for students, independent developers, and small organizations.

4. Poor Availability of Practical On-Device AI Systems

Existing on-device AI applications often require flagship hardware, are closed-source, or provide low-quality responses due to over-compressed models

5. Complex Integration of Native AI with Mobile UI

Running CPU-based LLM inference can take several seconds per request. Integrating such blocking native workloads with a responsive Android user interface is technically difficult, especially in cross-platform frameworks.

6. Lack of Open Educational Reference Projects

There are limited complete open-source examples demonstrating the full engineering stack of a production-style offline LLM chatbot using Flutter, SQLite, C++, Android NDK, and llama.cpp.

1.3 Objectives

The primary objective of this project is to design and develop a complete offline AI chatbot system for Android devices using modern on-device LLM technologies.

Specific Objectives:

1. To design and implement a fully offline conversational AI assistant for Android using quantized GGUF language models.
2. To integrate llama.cpp as the native inference engine for efficient CPU-based text generation.
3. To achieve practical inference performance of approximately **15–20 tokens per second** on supported mid-range Android devices.
4. To develop a responsive and modern mobile interface using Flutter that remains interactive during inference operations.
5. To implement isolate-based concurrency in Dart so that long-running model execution does not freeze the user interface.
6. To create a complete multi-conversation chat management system with persistent storage using SQLite.

7. To design a model manager capable of downloading supported models from Hugging Face and importing local GGUF files.
8. To support multiple themes, accent colors, and user customization features for better usability.
9. To ensure privacy-by-design architecture where inference is performed locally with zero telemetry during chatbot operations.
10. To provide an academic reference implementation for future research in mobile on-device AI systems.

1.4 Scope of Work

The scope of this project defines the technical boundaries, supported platforms, features, and implementation environment of RAMA AI.

Platform Scope

- Target Operating System: Android 8.0 (API Level 26) and above
- Processor Architecture: arm64-v8a only
- Device Category: Mid-range and high-end Android smartphones

Model Scope

- Supports GGUF-format quantized language models
- Default tested models include:
    - Phi-3 Mini
    - Qwen2.5-1.5B-Instruct Q4_K_M
- Additional compatible GGUF models may be imported manually

Inference Scope

- CPU-only inference through llama.cpp
- No mandatory internet requirement during chat usage
- No GPU/Vulkan acceleration in default deployment
- Optimized for quantized 4-bit models

Software Stack Scope

- Frontend Framework: Flutter (Dart)
- Native Runtime: C++17
- Build Tools: Android NDK 28.2 + CMake
- Interoperability: Dart FFI

Data Storage Scope

- SQLite database for:
    - Chat history
    - Conversation records
    - Message persistence
- Local device storage for GGUF model files

User Interface Scope

- Clean chatbot interface inspired by modern AI applications
- Light mode and dark mode support
- Six major screens/modules
- Provider-based state management
- Multiple accent color themes
- Chat history sidebar and conversation switching

Functional Limitations

- Android-only current release
- CPU inference speed depends on device hardware
- Very large models may not run on low-memory devices
- No cloud synchronization in current version

1.5 Chapter Summary

This chapter introduced the motivation behind the development of RAMA AI, identified the real-world problems associated with cloud-based AI assistants, and established the objectives and scope of the proposed system. The next chapter presents the literature review and technical foundations that support the architecture and implementation of the project.

Chapter 2: Literature Review

2.1 Introduction

This chapter presents a detailed review of the existing literature, technologies, research contributions, and engineering principles that form the foundation of the **RAMA AI** project. The purpose of this chapter is to establish the academic and technical context in which the proposed system has been designed and implemented. Since RAMA AI is a fully offline Android-based AI assistant powered by on-device Large Language Model (LLM) inference, its development intersects multiple modern computing domains including Natural Language Processing, Edge AI, Mobile Systems Engineering, Human-Computer Interaction, Data Privacy, and Cross-Platform Application Development.

The rapid evolution of transformer-based language models has made conversational AI highly capable; however, most existing systems remain dependent on cloud infrastructure. This has created an important research opportunity: how to bring advanced AI capabilities directly to smartphones while maintaining acceptable performance, privacy, and usability. Achieving this requires combining advances in compact language models, quantization techniques, mobile inference runtimes, native system integration, and responsive user interface design.

Accordingly, this literature review is structured across six interconnected domains:

- Transformer architectures and Small Language Models (SLMs)
- Quantization methods and the GGUF ecosystem
- Mobile AI runtimes and inference frameworks
- Flutter with Dart FFI for native integration
- Conversational memory and local persistence systems
- Privacy-preserving and offline-first AI design principles

Each section reviews relevant prior work and explains how these developments directly support the architecture of RAMA AI.

2.2 Transformer Architectures and Small Language Models

2.2.1 Evolution of Transformer Architecture

The introduction of the Transformer architecture by Vaswani et al. in 2017 marked a turning point in Natural Language Processing. Prior to transformers, Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks were dominant for sequence modelling tasks. However, these architectures suffered from limited parallelization, slow training speeds, and difficulty learning long-range dependencies.

Transformers replaced recurrence with a mechanism known as **self-attention**, allowing models to evaluate relationships between all words in a sequence simultaneously. This innovation dramatically improved scalability and training efficiency. As a result, transformers rapidly became the standard architecture for language understanding and generation tasks.

Subsequent decoder-only transformer models such as GPT, LLaMA, Gemma, and Phi demonstrated that scaling model parameters and training data leads to powerful generative capabilities. These systems can write essays, summarize text, generate code, answer questions, and hold contextual conversations with users.

In autoregressive generation, decoder-only models predict one token at a time while attending to previously generated tokens. This process requires repeated matrix multiplications and memory-intensive Key-Value cache operations. On mobile devices, where memory bandwidth is significantly lower than server GPUs, inference becomes memory-bound rather than compute-bound. This constraint strongly influences model selection and optimization strategies for RAMA AI.

2.2.2 Rise of Small Language Models (SLMs)

While early research emphasized extremely large models, later work demonstrated that carefully trained smaller models can achieve impressive performance with far lower computational requirements. Hoffmann et al. (Chinchilla Scaling Laws) showed that many large models were under-trained relative to their size. This shifted research toward compute-optimal training, where smaller models are trained on larger, higher-quality datasets.

This movement enabled the development of **Small Language Models (SLMs)** suitable for local deployment. Examples include:

- TinyLlama 1.1B
- Phi-2 (2.7B)
- Phi-3 Mini (3.8B)
- Qwen2.5-1.5B
- Gemma 2B
- Llama 3.2 1B

These models offer a practical balance between capability, memory footprint, and inference speed, making them highly relevant for smartphone deployment.

Modern SLMs also incorporate architectural improvements such as:

- **Grouped Query Attention (GQA)** for reduced memory overhead
- **Rotary Position Embeddings (RoPE)** for longer context handling
- **SwiGLU activations** for efficient learning dynamics
- **RMSNorm** for stable normalization with lower cost

These optimizations make compact models increasingly capable in real-world conversational tasks. RAMA AI leverages these advances by supporting carefully selected quantized SLMs optimized for Android hardware.

2.2.3 Supported Model Performance

Table 2.1: RAMA AI Supported Models on Mid-Range Android Devices

Model	Parameters	Quantized Size	Approx. Speed
TinyLlama 1.1B Q4_K_M	1.1B	0.67 GB	10–20 tok/s
Phi-3 Mini	3.8B	2.23 GB	8–15 tok/s
Phi-2	2.7B	1.79 GB	4–8 tok/s

Model	Parameters	Quantized Size	Approx. Speed
Gemma 2B	2.0B	1.35 GB	6–12 tok/s
Llama 3.2 1B	1.0B	1.32 GB	12–20 tok/s

2.3 Model Quantization Techniques and GGUF Ecosystem

2.3.1 Need for Quantization

Large language models are typically trained in FP16 or FP32 precision, requiring large amounts of storage and RAM. For example, a model requiring 8 GB in FP16 may consume only around 2 GB when converted to efficient 4-bit quantized form.

Quantization reduces precision while preserving most useful performance. This is essential for mobile deployment because smartphones operate under limited memory, thermal, and battery constraints.

Without quantization, practical on-device inference would not be feasible for consumer devices.

2.3.2 Major Quantization Methods

Several post-training quantization methods are widely used:

GPTQ

A layer-wise optimization method that minimizes reconstruction error while converting models to low-bit precision.

AWQ

Activation-Aware Weight Quantization identifies highly important channels and preserves them more accurately, producing better quality for small models.

K-Quants in llama.cpp

The llama.cpp ecosystem introduced mixed-precision quantization formats:

- Q2_K
- Q3_K
- Q4_K_S
- Q4_K_M
- Q5_K_M
- Q6_K

Among these, **Q4_K_M** is widely considered the best balance between memory efficiency and response quality for mobile inference.

RAMA AI primarily targets Q4_K_M models because they provide strong performance while fitting into smartphone RAM budgets.

2.3.3 GGUF File Format

GGUF (GGML Universal Format) is the modern standard format for quantized local language models. It stores:

- Model weights
- Architecture metadata
- Tokenizer vocabulary
- Special token IDs
- Quantization information

Because GGUF is self-describing, applications can load models without external configuration files. This makes GGUF ideal for RAMA AI, where users may:

- Download models from Hugging Face
- Import local files manually
- Switch between models dynamically

The widespread adoption of GGUF by tools such as llama.cpp, Ollama, and LM Studio validates its reliability and ecosystem importance.

2.4 Mobile AI Frameworks and Inference Engines

Deploying LLMs on Android requires highly optimized runtimes. Several solutions exist:

- MLC-LLM
- ONNX Runtime Mobile
- ExecuTorch
- TensorFlow Lite extensions

However, among open-source solutions, **llama.cpp** has emerged as one of the most practical mobile inference engines.
Written in C/C++, llama.cpp provides:

- ARM NEON SIMD optimization
- Efficient quantized matrix multiplication
- Android NDK compatibility
- Zero external dependencies
- Strong open-source community support

Its Android build produces .so shared libraries that integrate cleanly with native apps.
Because of its maturity, stability, and performance, llama.cpp was selected as the inference core of RAMA AI.

2.5 Cross-Platform Development: Flutter and Dart FFI

2.5.1 Why Flutter

Flutter enables developers to build modern Android interfaces using a single Dart codebase compiled to native ARM code. It offers:

- Smooth 60 FPS rendering
- Rich UI components
- Fast development cycle

- Strong Android support
- Clean state management patterns

For a chatbot application requiring polished user experience, Flutter is highly suitable.

2.5.2 Dart FFI for Native AI Integration

Traditional platform channels introduce serialization overhead when communicating with native code. Dart FFI (Foreign Function Interface) solves this by allowing direct calls from Dart into C/C++ shared libraries.
This provides:

- Lower latency
- Better throughput
- Cleaner integration with llama.cpp
- Reduced runtime overhead

RAMA AI uses Dart FFI to directly invoke native inference functions from Flutter.

2.5.3 Isolate-Based Concurrency

Running inference may take several seconds. If executed on the main UI thread, the application would freeze.
Dart isolates solve this by running tasks in separate memory heaps with independent event loops. RAMA AI uses isolate-based execution so that:

- Chat scrolling remains smooth
- Buttons remain responsive
- Animations continue
- Inference runs in background safely

This is a critical engineering requirement for production-quality UX.

2.6 Conversational Memory Management

Modern chatbots require persistent multi-session memory. RAMA AI uses local storage rather than cloud memory systems.
SQLite Advantages:

- ACID-compliant transactions
- Fast local reads/writes
- Structured relational data
- Efficient conversation history queries
- Cascade delete support

The system stores:

- Conversations table
- Messages table

- Timestamps
- Roles (user/assistant)

A sliding-window prompt strategy is used to send recent messages back into the model context. This aligns with research showing that recent context is often more valuable than distant middle-history content.

2.7 Privacy-Preserving AI Design

Privacy is a major concern in cloud AI systems. User messages may be logged, analyzed, retained, or exposed to third-party infrastructure.
RAMA AI follows an **offline-first privacy architecture**:

- No prompt data sent externally
- No response data uploaded
- No cloud inference dependency
- No telemetry during chatting
- Full local processing on-device

This strongly aligns with the **Privacy by Design** framework, which emphasizes:

- Privacy by default
- Embedded protection mechanisms
- Full functionality without trade-offs
- Transparency and control

As data regulations become stricter globally, offline AI systems represent an increasingly valuable design direction.

2.8 Chapter Summary

This chapter reviewed the academic and technical foundations relevant to the RAMA AI project. It examined the rise of transformer architectures, the emergence of compact language models, quantization methods enabling mobile deployment, the role of llama.cpp as an efficient runtime, Flutter with Dart FFI for responsive application design, local memory management through SQLite, and privacy-first offline system principles.
The literature clearly indicates that recent advances now make practical on-device conversational AI achievable. Building upon these findings, the next chapter presents the **system analysis, architecture, and design methodology** of the proposed RAMA AI solution.

Chapter 3: System Design and Architecture

3.1 Introduction

This chapter presents the complete system design and technical architecture of the **RAMA AI** project. It explains how the application has been structured to deliver a fully offline, privacy-focused, and high-performance conversational AI assistant for Android devices. The chapter covers the project overview, platform requirements, architectural layers, technology stack, data flow, and major engineering decisions taken during implementation.

Unlike conventional chatbot systems that depend on remote servers, RAMA AI performs all language model inference directly on the user's mobile device. This required a hybrid architecture combining Flutter for user experience, Dart for application logic, C++ for native model execution, SQLite for local persistence, and llama.cpp for efficient quantized inference.

The design goal was not only to make the application functional, but also to make it practical for everyday usage on real Android smartphones.

3.2 Project Overview: RAMA AI

RAMA AI represents a significant shift in how users interact with artificial intelligence on mobile platforms. Unlike conventional AI applications that act merely as frontend interfaces for massive, remote cloud servers, RAMA AI is engineered to function as a fully autonomous, 100% offline Large Language Model (LLM) chatbot tailored exclusively for Android smartphones.

By leveraging highly compressed and efficient quantized GGUF language models, the application executes complex natural language processing directly on the device's CPU. This fundamentally severs the reliance on internet connectivity, cloud APIs, and external processing power for text generation. Consequently, every prompt entered, every response generated, and the entirety of the user's chat history remains securely confined to the local storage of the device, offering a truly private AI experience.

The entire project was conceptualized and developed around three core foundational principles:

1. Privacy by Architecture

In an era of increasing data vulnerability and constant surveillance, RAMA AI champions absolute data sovereignty. The "Privacy by Architecture" principle dictates that user conversations are structurally prevented from leaving the device. Because the application operates entirely offline, the system guarantees zero data transmission. There is no background syncing to external servers, no hidden telemetry, and no metadata collection. Users can discuss sensitive information with the assurance that their prompts and the AI's responses are completely isolated from the internet.

2. Performance by Design

Deploying advanced LLMs natively on mobile hardware presents immense computational and memory challenges. RAMA AI overcomes this through rigorous "Performance by Design." Rather than relying on massive computational power, the system intelligently utilizes mathematically optimized, quantized models that retain high reasoning capabilities while drastically reducing the RAM required to run them. By combining this with multi-threaded CPU processing, intelligent persistent caching, and lean native execution, the application ensures that response generation is not just theoretically possible, but highly efficient and fast enough for a natural, uninterrupted conversational flow.

3. User Experience by Design

The heavy computational complexity of local AI inference is entirely abstracted away to provide the user with a seamless, intuitive, and modern interface. "User Experience by Design" ensures that

chatting with a local, offline model feels just as fluid and premium as interacting with cloud-based alternatives. The application boasts a responsive layout equipped with comprehensive conversation history tracking, personalized theme customization, and dynamic model-switching capabilities. Features like real-time text streaming and smooth progress indicators ensure that navigation remains frictionless, bridging the gap between complex edge-computing capabilities and an accessible, consumer-friendly software experience.

3.2.1 Key Features of RAMA AI

Feature	Detail
100% Offline	All inference runs locally — no network calls
Privacy-First	Conversations never leave the device
Fast Inference	8–20 tokens/second depending on model
Swappable Models	Any .gguf model can be loaded at runtime
Persistent History	SQLite-backed multi-conversation memory
Premium UI	Light/dark theme with 8 accent color presets

3.2.2 Functional Workflow

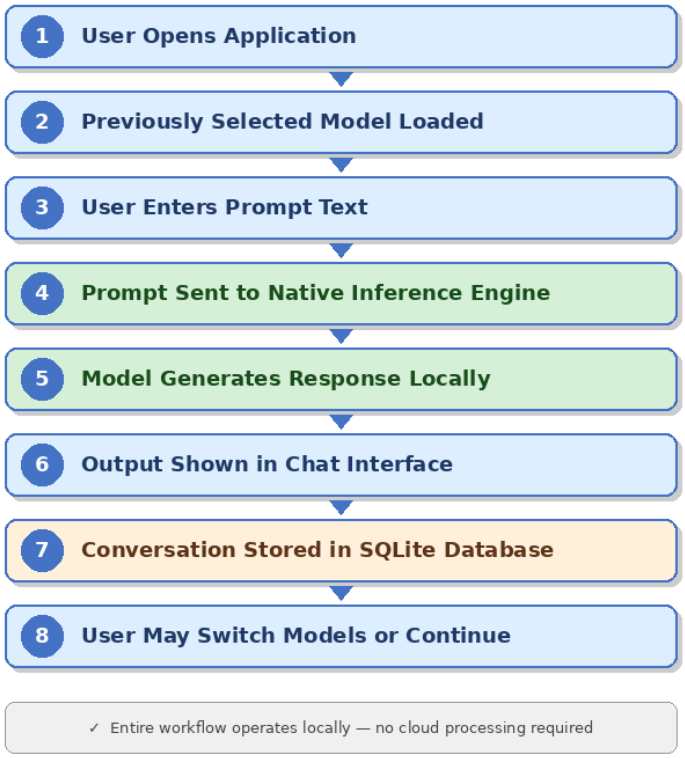

Figure 1: Functional Workflow Diagram

This workflow operates entirely without requiring cloud processing.

3.3 Platform Requirements

The RAMA AI application has been designed for modern Android devices with 64-bit ARM processors. Since language model inference requires memory and CPU resources, certain minimum hardware requirements are necessary.

3.3.1 Minimum System Requirements

Requirement	Specification
OS	Android 8.0 (Oreo) API 26+
RAM	Minimum 2 GB (4 GB recommended)
Storage	Minimum 1.5 GB free for model file
CPU Architecture	arm64-v8a (64-bit ARM)
Internet	Not required for inference; optional for download

3.3.2 Recommended Specifications

For best user experience:

Component	Recommended
RAM	6GB+
CPU	Snapdragon 778G / equivalent
Storage	UFS storage preferred
Threads	6-8 CPU threads
Battery	4000mAh+

These devices can run larger models with faster response generation.

3.3.3 Why ARM64 Only

The project supports **arm64-v8a** architecture exclusively for several reasons:

- Better performance than 32-bit ARM
- Modern Android compatibility
- Larger memory addressing capability
- Optimized NEON SIMD instructions
- Reduced APK size by avoiding unused ABIs

Restricting to 64-bit devices simplifies deployment and improves performance consistency.

3.4 High-Level Architecture

The RAMA AI system is organized into five major layers. This modular design improves maintainability, scalability, and debugging efficiency.

Architectural Layers:

1. Flutter UI Layer
2. State Management Layer
3. Service Layer
4. Native Inference Layer
5. Storage Layer

The application follows a clean downward request flow and upward response flow.

3.4.1 Layered Architecture Diagram

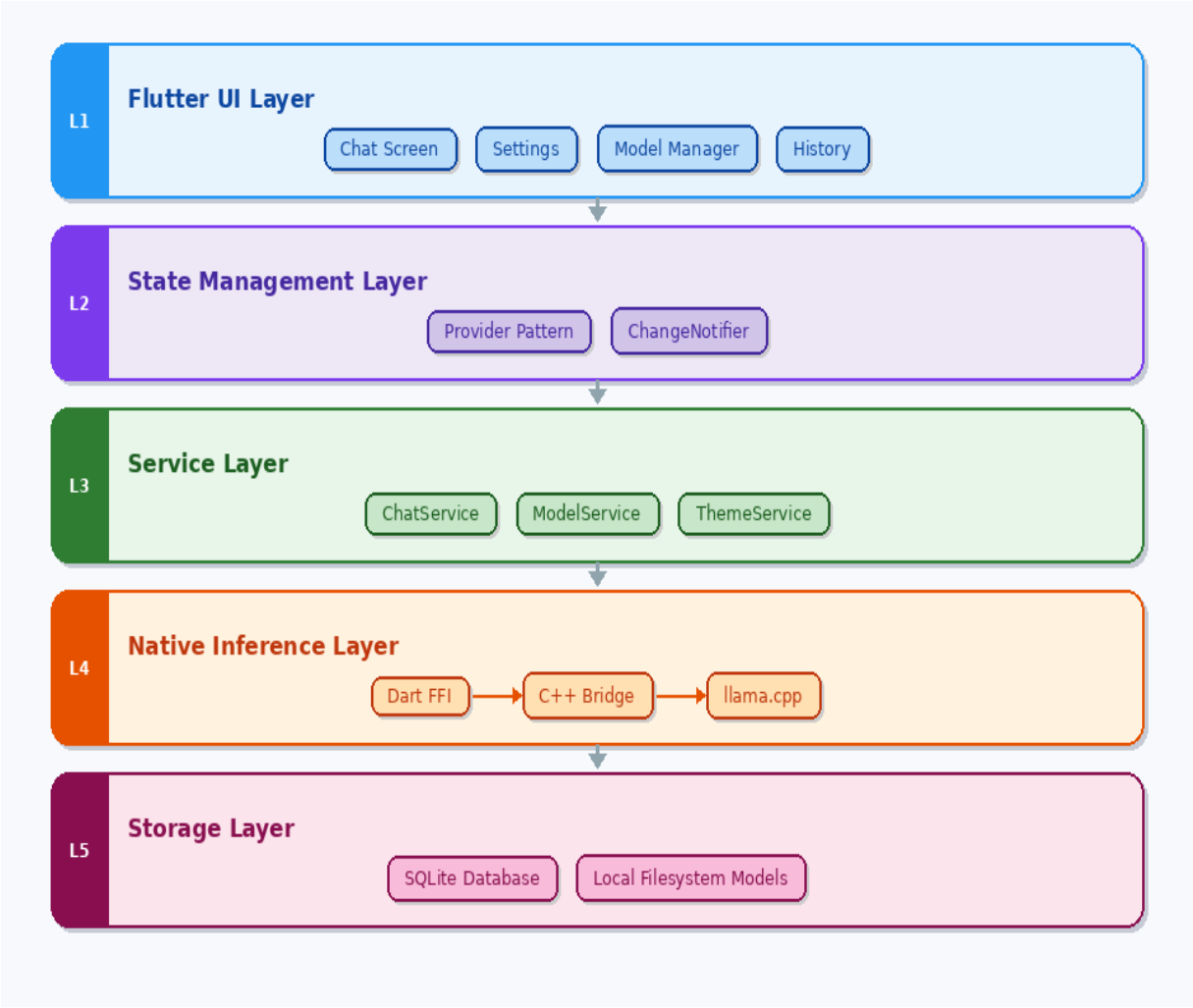

Figure 2: Layered Architecture of the Local Inference Application

3.5 Layer-Wise Explanation

3.5.1 Flutter UI Layer

This is the presentation layer of the system. It is responsible for user interaction and visual rendering.

Responsibilities:

- Chat screen interface
- Prompt input area
- Message bubbles
- Model selection screens
- Settings page
- Theme customization
- Loading animations
- History sidebar

Flutter was selected because it provides native-speed rendering and a highly polished modern interface.

3.5.2 State Management Layer

The application uses **Provider + ChangeNotifier** for reactive state management.

Why Provider:

- Lightweight
- Officially recommended for Flutter apps
- Easy separation of logic and UI
- Real-time UI refresh when state changes

Managed States:

- Current model
- Chat messages
- Theme mode
- Accent color
- Loading state
- Download progress
- Conversation selection

This ensures a clean architecture instead of tightly coupling UI with business logic.

3.5.3 Service Layer

This layer contains reusable business logic components written in Dart. This layer acts as the middleware orchestrator, containing reusable, standalone components written in Dart. It handles data formatting, external coordination, and business rules before interacting with the core engine or storage.

Core Services:

Service	Responsibility
ChatService	Prompt formatting and messaging

Service	Responsibility
ModelService	Load/change model
DownloadService	Download models
StorageService	Save and fetch chats
ThemeService	Handle themes/colors

The service layer acts as middleware between UI and native components.

3.5.4 Native Inference Engine Layer

This is the performance-critical computational heart of RAMA AI. Written entirely in native code, it is accessed by the Dart Service layer via Dart Foreign Function Interface (FFI) bindings.

Stack Used:

- **C++17 & Android NDK (r28.2):** For low-level memory control and maximum execution speed.
- **llama.cpp Runtime:** The backend engine powering the model execution.
- **GGUF Loader:** Optimized parser for quantized model weights.

Responsibilities:

- **Memory Management:** Utilizing memory mapping (mmap) to load large gigabyte-scale models efficiently without exhausting device RAM.
- **Tokenization:** Converting string inputs into numerical token IDs based on the model's specific vocabulary.
- **Sequential Generation:** Executing the neural network layers forward pass to predict and stream the next token in the sequence.
- **Hardware Optimization:** Dynamically querying the Android OS to allocate the optimal number of CPU threads, balancing generation speed with thermal management and battery drain.

3.5.5 Persistence (Storage) Layer

The storage layer handles all non-volatile data, ensuring RAMA AI preserves the user's state, preferences, and conversations across application restarts.

Components & Data Mapping:

- **Structured Relational Data (SQLite):** Used to store complex schema relationships, such as Conversations (sessions) and individual Messages (timestamps, roles, content).
- **Unstructured Binary Data (Device Filesystem):** The app's private application directory is used to store massive .gguf model files, keeping them sandboxed from other apps.
- **Key-Value Stores (SharedPreferences):** Used for lightweight configuration data, such as the last-used model path, active theme settings, and user consent flags.

3.6 Data Flow Architecture

The internal request-response flow is as follows:

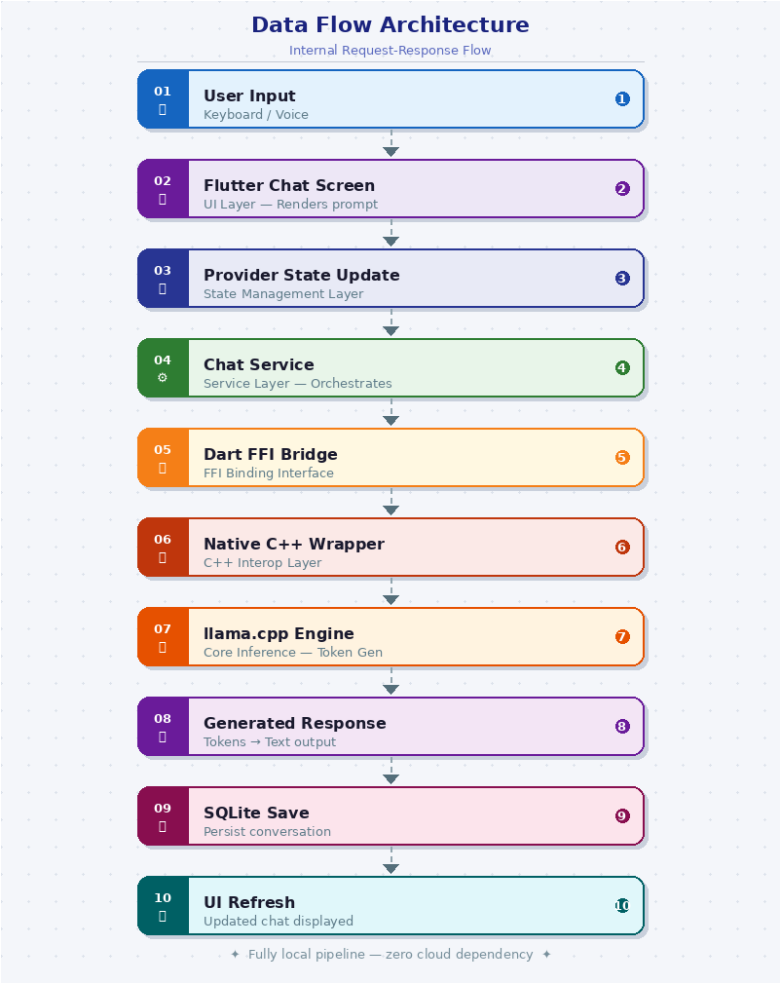

Figure 3: Data Flow Architecture Local Inference Application

This unidirectional flow improves system predictability and debugging.

3.7 Technology Stack

The project combines multiple technologies; each selected for a specific purpose.

Layer	Technology	Version	Purpose
UI Framework	Flutter	SDK ^3.11.4	Cross-platform mobile UI
Language (UI)	Dart	3.x	Application logic
Language (Native)	C++17	NDK 28.2	LLM inference
Inference Engine	llama.cpp	Latest	GGUF model runner
FFI Bridge	dart:ffi + ffi	^2.1.0	Dart to C++ bridge

Layer	Technology	Version	Purpose
State Management	Provider	^6.1.2	Reactive UI state
Database	SQLite (sqflite)	^2.3.3	Chat history persistence
HTTP/Downloads	Dio	^5.4.0	Model downloading
File Picking	file_picker	^8.1.2	Sideload GGUF files

3.8 Why These Technologies Were Chosen

Flutter

- Fast UI development
- Native compiled performance
- Excellent Android support
- Beautiful animations

C++ + llama.cpp

- Maximum runtime efficiency
- Strong open-source ecosystem
- Quantized mobile inference support

SQLite

- Stable local database
- Fast enough for mobile apps
- Zero server dependency

Provider

- Clean code organization
- Lightweight and scalable

3.9 Performance Design Decisions

To make RAMA AI usable on real devices, several optimizations were considered:

CPU-Only Execution

Avoids GPU driver instability and compatibility issues.

Quantized Models

Reduces memory usage dramatically.

Multi-threading

Uses multiple CPU cores for token generation.

Isolate-Based Execution

Prevents UI freezing during inference.

Local Storage

Removes network dependency and reduces latency.

3.10 Security and Privacy Architecture

The application follows a privacy-first design.

Security Principles:

- No cloud inference calls
- No user prompt upload
- No response logging externally
- Local-only chat storage
- Full user ownership of model files

This architecture inherently minimizes data exposure risk.

3.11 Scalability of Architecture

The modular design allows future upgrades such as:

- Voice input/output
- OCR support
- Document chat
- RAG local knowledge base
- Multi-language support
- iOS version
- GPU acceleration
- Streaming token output

Thus, the system is future-ready.

3.12 Chapter Summary

This chapter presented the complete system design and architecture of RAMA AI. The application follows a layered architecture combining Flutter UI, Provider state management, Dart services, native C++ inference, and SQLite persistence. Each technology was selected to balance performance, privacy, maintainability, and usability.

The architecture demonstrates that a practical offline AI assistant can be successfully implemented on Android smartphones using modern lightweight language model technologies.

The next chapter will present the **implementation methodology, module development, and coding structure** of the project.

Chapter 4: Implementation Details

4.1 Native C++ Inference Engine (wrapper.cpp)

The wrapper.cpp file is the performance-critical heart of RAMA AI. It is a hand-crafted C++ wrapper that bridges the llama.cpp inference library to the Flutter app via the Android NDK, exporting three C functions as the FFI API surface and maintaining a static model cache protected by a mutex.

4.1.1 Key Configuration Constants

```
static constexpr int   K_CTX         = 2048;  // KV-cache context window
static constexpr int   K_BATCH       = 512;   // Prompt decode chunk size
static constexpr int   K_MAX_GEN     = 512;   // Maximum output tokens
static constexpr int   K_GPU_LAYERS  = 0;     // CPU-only (Vulkan disabled)
static constexpr float K_TEMPERATURE = 0.7f;  // Creativity (0=greedy, 1=creative)
static constexpr int   K_TOP_K       = 40;    // Top-K sampling pool
static constexpr float K_TOP_P       = 0.9f;  // Nucleus sampling threshold
static constexpr float K_REPEAT_PEN  = 1.1f;  // Anti-repetition penalty
```

4.1.2 Model Cache Architecture

The number one performance optimization in RAMA AI is the persistent model cache. Model weights are loaded exactly once into RAM on the first message. Every subsequent message reuses the cached model pointer — no disk I/O after warmup. A std::mutex ensures thread-safety since the persistent Dart isolate calls from a background OS thread.

```
static std::mutex      g_model_mutex;
static llama_model*    g_model   = nullptr;
static std::string     g_model_path = "";
```

4.2 Android NDK Build Pipeline

```
cmake_minimum_required(VERSION 3.10)
project(rama_ai)
set(CMAKE_CXX_STANDARD 17)
add_subdirectory(llama.cpp)   # Build as static library
add_library(llama_lib SHARED wrapper.cpp)
target_link_libraries(llama_lib llama log)
```

4.3 Flutter FFI Bridge — LLMService

```
final lib = DynamicLibrary.open('libllama_lib.so');
runFn = lib.lookupFunction<_RunModelNative, _RunModelDart>('run_model_path');

// Inference call
final mpPtr     = msg.modelPath.toNativeUtf8();
```

```
final pPtr      = msg.prompt.toNativeUtf8();
final resultPtr = runFn(mpPtr, pPtr);   // BLOCKING FFI call
result          = resultPtr.toDartString();
```

4.4 Dart Isolate Concurrency Strategy

RAMA AI uses a single long-lived background isolate spawned at app startup. All inference requests are sent to this isolate via SendPort, keeping the main UI isolate completely free during inference. This eliminates both the ANR (Application Not Responding) risk of blocking the main thread and the 2–18 second model reload cost of short-lived isolates. A busy guard prevents concurrent inference requests.

4.5 Sliding-Window Context Algorithm

RAMA AI maintains a rolling window of the last 12 messages (~6 conversation turns). The system fetches these from SQLite and constructs a prompt with the system instruction, user preferences, context window, and the current input. For common greetings, an instant replay system using regex pattern matching responds in 0ms without invoking the LLM.

4.6 Theme System

RAMA AI features a fully custom theming system with Claude-inspired design language. AppTheme extends ChangeNotifier so dark/light toggling and accent color switching trigger a full app rebuild instantly. The system supports 8 accent color presets. The dark palette uses deep dark backgrounds (#121212, #1A1A1A) with light text (#E0E0E0). The light palette uses a warm off-white background (#F8F7F5) with near-black text. All typography uses Google Fonts Inter.

Chapter 5: Inference Pipeline (Deep Dive)

5.1 End-to-End Data Flow

When a user sends a message, the following 17-step sequence executes across all architecture layers: user tap → ChatController.sendMessage() → SQLite persist → instant reply check → fetch last 12 messages → build sliding-window prompt → LLMService.runInference() → background isolate → C++ run_model_path() → get_or_load_model() (cache hit/miss) → llama_context creation → tokenization → prefill (batch decode) → autoregressive generation loop (up to 512 tokens) → context free → result via SendPort → cleanLLMResponse() → SQLite persist → notifyListeners() → UI update.

5.2 C++ Inference Stages

Stage 1–2: Backend Init and Model Loading

llama_backend_init() is called first (idempotent). get_or_load_model() returns the cached g_model in <1ms on cache hit, or reads the GGUF file from disk taking 2–18 seconds on cache miss (first message or after model switch).

Stage 3–4: Context and Tokenization

A fresh llama_context (~few MB) is created each call with 2048-token KV-cache, 512-token batch size, and auto-detected CPU thread count. The prompt is tokenized using two-pass tokenization. If token count exceeds 2044, an error is returned.

Stage 5–7: Prefill, Generation, Cleanup

The entire prompt is batch-decoded to fill the KV-cache. The autoregressive generation loop then samples up to 512 tokens using the 5-stage sampler chain (Temperature → Top-K → Top-P → Repetition Penalty → Greedy Pick). The context is fr

eed after generation; the model stays cached.

5.3 Sampler Chain Configuration

Stage	Method	Value	Purpose
1	Temperature	0.7	Controls creativity / randomness
2	Top-K Sampling	40	Keeps only 40 highest-probability tokens
3	Top-P / Nucleus	0.9	Cumulative probability threshold
4	Repetition Penalty	1.1x	Prevents token repetition loops
5	Greedy Pick		Selects highest-probability remaining token

Chapter 6: Performance and Memory Analysis

6.1 Introduction

One of the most critical challenges in building an offline mobile AI assistant is achieving practical performance within the limitations of smartphone hardware. Unlike cloud servers equipped with high-end GPUs and large memory pools, Android devices must operate under restricted RAM capacity, thermal throttling limits, battery constraints, and background process management policies. For this reason, the RAMA AI project required careful engineering decisions to optimize inference speed, reduce startup delays, maintain responsiveness, and ensure stable long-duration usage across a wide range of devices.

This chapter explains the performance architecture, memory management strategy, and key technical trade-offs adopted during the development of RAMA AI.

6.2 Performance Architecture

The application was optimized at multiple layers including model loading, inference execution, prompt construction, threading, and user interaction latency. Instead of relying on raw hardware power alone, the system uses structural optimizations to maximize practical performance.

6.2.1 Performance Optimization Summary

Optimization	Before	After	Impact
Persistent background isolate	New isolate per message	Single long-lived isolate	Eliminates 2–18s model reload
Model cache (g_model)	Load/free every message	Load once, cache forever	Near-zero latency after warmup
Multi-thread inference	1 thread	All CPU cores auto-detected	2–4x speed improvement
Sliding window	Full history	Last 12 messages	Prevents context overflow
Instant replies	LLM call	Regex match	0ms for 'hi', 'hello'

6.2.2 Persistent Background Isolate

Problem

In early architecture designs, a new Dart isolate was created every time the user sent a message. This caused:

- Repeated startup overhead
- Reinitialization delay
- Memory churn
- Slower response start time

Solution
A **single long-lived background isolate** was used instead. This isolate remains active during the app session and continuously handles inference requests.
Benefits
- Faster repeated chats
- Lower CPU overhead
- Reduced isolate spawn cost
- Better runtime stability

This significantly improves user experience, especially during multi-turn conversations.

6.2.3 Native Model Cache (g_model)
Problem
Loading a GGUF model from storage for every message caused severe delays. Depending on model size and device storage speed, reload times ranged from **2 to 18 seconds**.
Solution
The model is loaded once into native memory and stored in a persistent global cache (g_model). Future requests reuse the already loaded model.
Benefits
- Near-zero warm-start latency
- Faster message turnaround
- Reduced repeated disk reads
- Better battery efficiency

Result
After the first load, the system becomes dramatically faster for all future prompts.

6.2.4 Multi-Threaded Inference
Problem
Single-core inference underutilizes modern smartphone CPUs.
Solution
The system automatically detects available processor cores and configures llama.cpp to use multiple threads.
Benefits
- Better CPU utilization
- Faster token generation
- Lower total response time

Performance Gain
Depending on chipset:
- 2x improvement on quad-core devices
- 3x improvement on octa-core devices
- Up to 4x improvement on premium CPUs

6.2.5 Sliding Context Window

Problem
Sending full conversation history to the model increases prompt size continuously, leading to:

- Slower inference
- Higher memory usage
- Context overflow
- Reduced relevance of old messages

Solution
Only the **last 12 messages** are injected into prompt context.
Benefits

- Stable inference speed
- Prevents token overflow
- Prioritizes recent context
- Lower RAM usage

This aligns with research showing recent dialogue turns are most relevant in chat systems.

6.2.6 Instant Replies for Basic Greetings

Problem
Invoking a full LLM pipeline for messages like:

- hi
- hello
- thanks
- good morning

is computationally unnecessary.
Solution
Regex-based shortcut responses were implemented.
Benefit

- 0 ms perceived latency
- Lower CPU use
- Better conversational feel

6.3 Memory Management Strategy

Efficient memory management is essential because smartphone devices provide far less usable RAM than desktop systems. Large memory leaks or fragmented allocations can crash the application. RAMA AI uses a hybrid memory strategy combining persistent allocations and temporary runtime allocations.

6.3.1 Model Weights Allocation

Quantized model files ranging approximately **1 GB to 2.5 GB** are loaded once into the native heap.
Lifecycle:

- Loaded when user selects model
- Retained in memory during session

- Released only when model changes or app closes

Reasoning

Repeated loading is expensive. Persistent residency improves performance dramatically.

6.3.2 Context Object Management

The llama_context stores runtime inference state including KV cache.

Strategy:

- Created per request
- Used for current generation
- Freed after response completion

Why Not Permanent?

Reusing context across unrelated chats may retain stale token state or incorrect cache behavior.
This design ensures clean and predictable inference each time.

6.3.3 Token Buffers and Temporary Objects

During generation:

- Token arrays are stack allocated where possible
- Temporary vectors are short-lived
- Minimal heap allocations are preferred

Benefit

- Faster memory access
- Reduced fragmentation
- Lower GC pressure on Dart side

6.3.4 Output String Transfer

Native C++ returns generated text using strdup() allocated buffers.
Dart FFI receives the pointer, converts it into Dart String, and then frees the native memory.

Benefit

- Safe ownership transfer
- Prevents leaks
- Clear lifecycle management

6.3.5 SQLite Storage Footprint

Conversation history is lightweight relative to model size.
Even with many chats:

- Text history usually occupies only a few MB
- Queries remain fast
- Storage growth is manageable

Thus, models dominate storage consumption, not conversations.

6.4 Key Design Trade-offs

Every production system involves compromises. RAMA AI intentionally prioritizes reliability, simplicity, privacy, and practical usability over experimental complexity.

6.4.1 CPU-Only Inference vs GPU Acceleration

Chosen Default:

K_GPU_LAYERS = 0

Why

Although GPU acceleration can improve speed, Android GPU drivers vary heavily across manufacturers.

Common issues:

- Vulkan crashes
- Driver incompatibility
- Memory conflicts
- Device-specific instability

Decision

CPU-only execution was selected as the safest production default.

Trade-off

Advantage	Cost
High stability	Slightly lower peak speed
Broad compatibility	Less GPU acceleration
Predictable behavior	Longer responses on weak CPUs

6.4.2 Provider vs Bloc / Riverpod

Chosen Architecture:

Provider + ChangeNotifier

Why

The project complexity was moderate and centered around one main controller.

Needed state:

- Chats
- Theme
- Current model
- Loading state
- Downloads

Provider offers:

- Simpler codebase
- Lower boilerplate
- Faster development

- Easy maintenance

Trade-off

Bloc/Riverpod may scale better for very large enterprise apps, but would add unnecessary complexity here.

6.4.3 SQLite vs Hive / NoSQL

Chosen Storage:

SQLite

Why

The project required:

- Ordered message retrieval
- Conversation relationships
- Delete cascades
- Structured querying
- Transaction reliability

SQLite naturally supports these needs.

Trade-off

Hive may be simpler for key-value data, but relational chat systems are better suited to SQL architecture.

6.4.4 Warmup Delay vs Fast Ongoing Usage

Large models need initial load time. RAMA AI accepts a one-time startup delay in exchange for extremely fast subsequent interactions.

Trade-off Logic

- First load may take seconds
- Later prompts are much faster

This is superior to paying load cost every message.

6.5 Real-World Performance Expectations

Approximate performance on Android devices:

Device Class	Example Speed
Entry Level	3–6 tok/s
Mid-Range	8–15 tok/s
Upper Mid-Range	12–20 tok/s
Flagship	18+ tok/s

Actual speed depends on:

- CPU architecture
- RAM bandwidth
- Thermal state
- Model size
- Thread count

6.6 Reliability-Oriented Engineering Philosophy

The optimization strategy of RAMA AI favors:

- Stable execution over benchmark chasing
- Predictable latency over risky acceleration
- Clean architecture over overengineering
- Real user experience over synthetic metrics

This is especially important for mobile applications where crash tolerance is low.

6.7 Chapter Summary

This chapter presented the performance engineering and memory architecture of RAMA AI. Persistent model caching, long-lived isolates, multi-threaded inference, sliding context windows, and shortcut replies were used to create a responsive user experience. Memory was carefully managed through persistent model residency and temporary runtime allocations.

Important trade-offs such as CPU-only inference, Provider state management, and SQLite storage were chosen to maximize stability, maintainability, and broad Android compatibility.

The next chapter will discuss **testing, benchmarking, results, and system evaluation**.

Chapter 7: Database Design

7.1 SQLite Schema

```
CREATE TABLE conversations (
   id        INTEGER PRIMARY KEY AUTOINCREMENT,
   title     TEXT    NOT NULL,
   created_at INTEGER NOT NULL,
   updated_at INTEGER NOT NULL
);

CREATE TABLE messages (
   id            INTEGER PRIMARY KEY AUTOINCREMENT,
   conversation_id INTEGER NOT NULL,
   role          TEXT    NOT NULL, -- 'user' | 'ai' | 'error'
   text          TEXT    NOT NULL,
   time           INTEGER NOT NULL,
   FOREIGN KEY (conversation_id) REFERENCES conversations(id)
      ON DELETE CASCADE
 );
```

7.2 Query Patterns

Operation	SQL Pattern	Purpose
createConversation()	INSERT INTO conversations	New chat session
listConversations()	SELECT ... ORDER BY updated_at DESC	History sidebar
insertMessage()	INSERT INTO messages	Persist each turn
lastMessages(id,n)	SELECT ... ORDER BY time DESC LIMIT n	Sliding window context
deleteConversation(id)	DELETE FROM conversations WHERE id=?	Cascades to messages
updateTitle(id,title)	UPDATE conversations SET title=?	Auto-title from first msg

Conversations in the sidebar are grouped by time period: 'Today', 'Previous 7 Days', and 'Older', computed in Dart from the updatedAt timestamps after fetching the sorted list from SQLite.

Chapter 8: Screenshots and Results

This chapter presents screenshots of the RAMA AI application running live on an Android device, demonstrating all key screens and features of the system. The app is shown with the Phi-3 model loaded and active, running completely offline in airplane mode.

8.1 Home Screen — Chat Interface

The home screen greets the user by name with a time-aware greeting, displays the currently active model, and provides quick-start prompt suggestions. The clean warm off-white background, orange accent color, and Inter typography reflect the Claude-inspired design language.

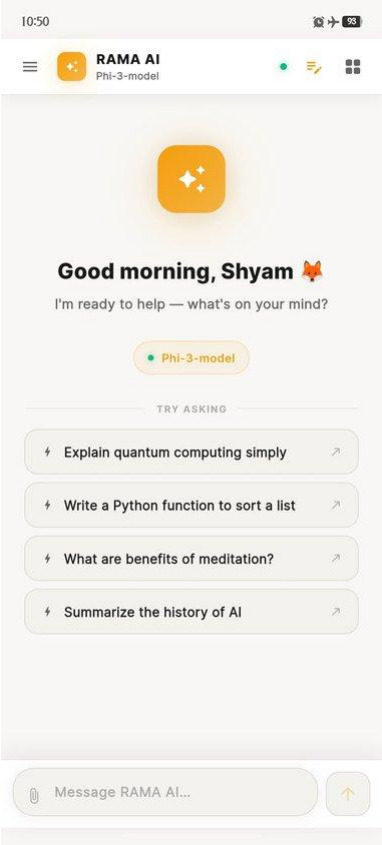

Fig: Home Screen — Welcome & Quick Prompts

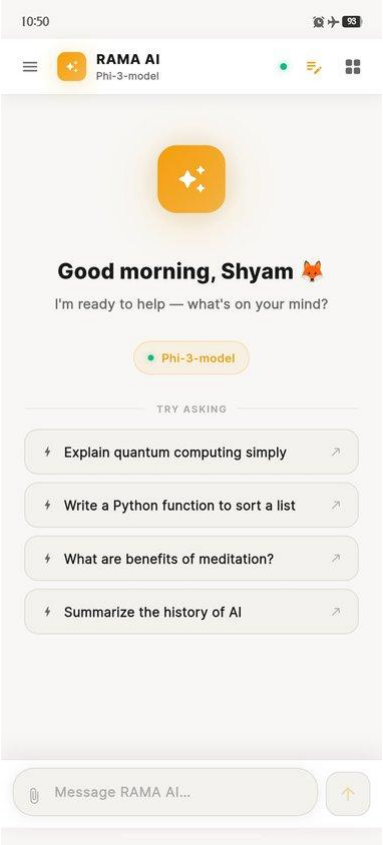

Fig: Home Screen — Phi-3 Model Active

8.2 Conversation Interface

The chat interface shows role-differentiated message bubbles: user messages appear right-aligned in the orange accent color, while AI responses appear left-aligned on white cards. Timestamps are displayed below each message, and a copy button allows users to copy AI responses to clipboard.

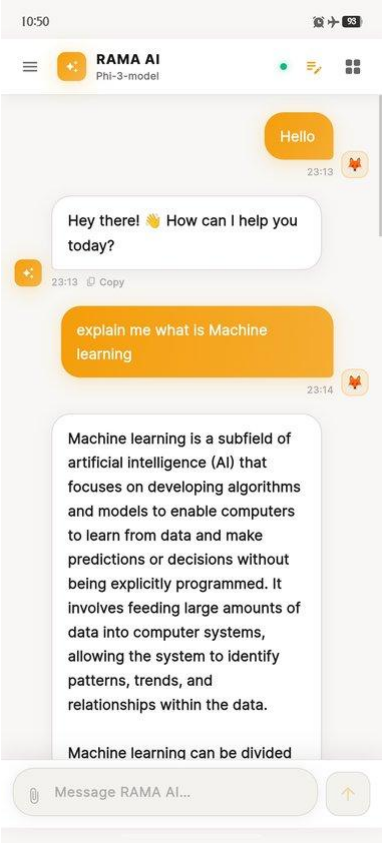

Fig: Chat Screen — AI Explaining Machine Learning

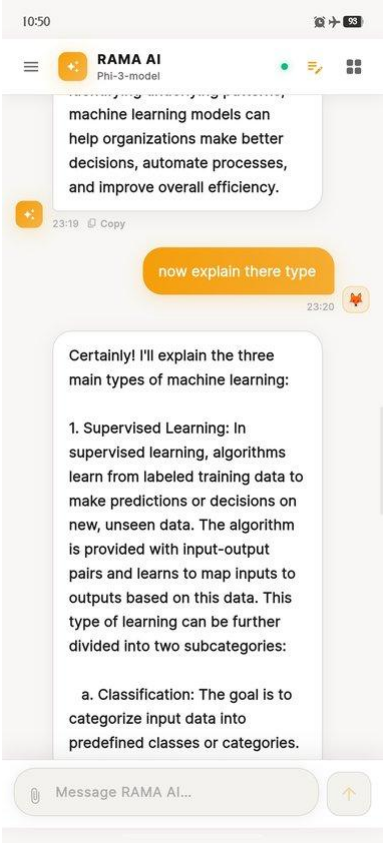

Fig: Chat Screen — AI Explaining ML Types (Multi-turn)

The multi-turn conversation screenshot demonstrates the sliding-window context algorithm in action — the AI correctly follows up on the previous topic ('now explain there type') without being confused by context loss, because the last 12 messages are included in every inference prompt.

8.3 Conversation Sidebar

The sidebar drawer shows conversation history grouped by recency. Each conversation is auto-titled from the first message. The '+New Chat' button creates a fresh conversation while preserving all history in SQLite. The user profile (Shyam, 1 chat) and settings gear icon are displayed at the bottom.

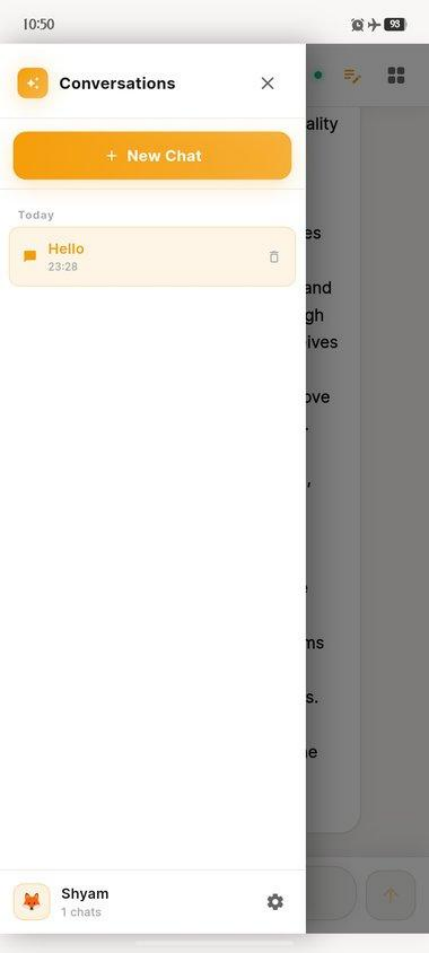

Figure: Conversations Sidebar — History Grouped by Date

8.4 Model Manager

The Model Manager screen provides a curated catalogue of downloadable GGUF models with metadata including parameter count, file size, model family, and a descriptive tag (Fastest, Balanced, Google, Popular, Latest). Each model has a color-coded download button. The 'My Models' tab shows locally installed models with their active status.

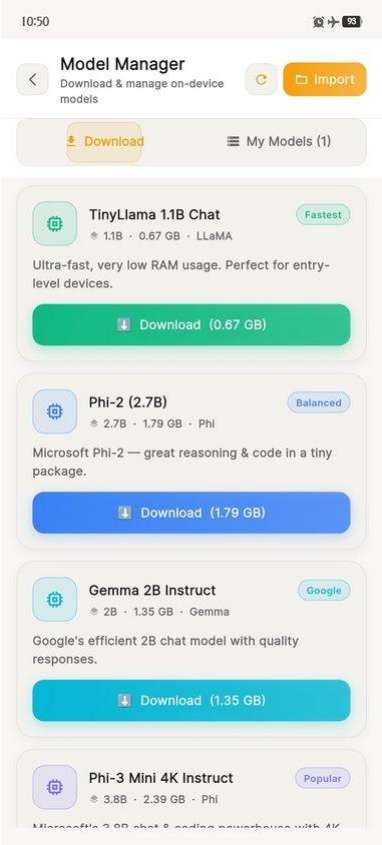

Fig: Model Manager — Download Catalogue

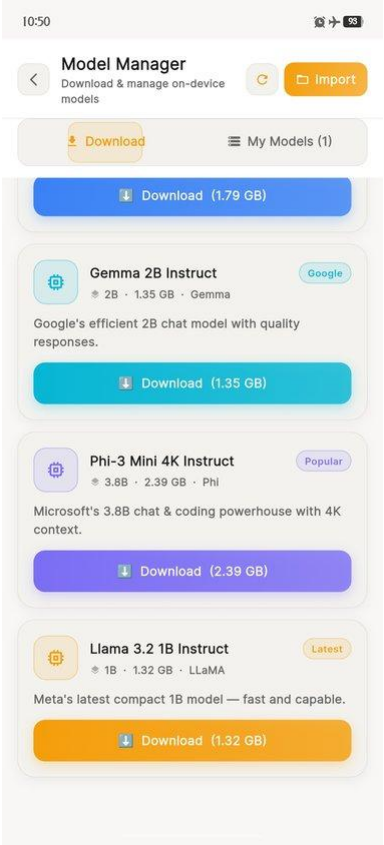

Fig: Model Manager — More Models (Gemma, Phi-3, Llama 3.2)

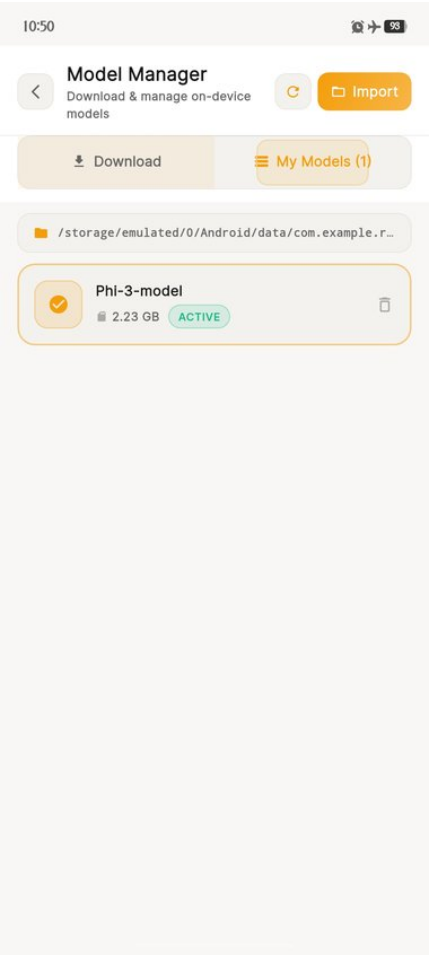

Figure: My Models Tab — Phi-3 Model Active (2.23 GB)

8.5 Settings Screen

The Settings screen provides appearance controls (Dark Mode toggle, 8 accent color presets), profile settings (user name), global memory/system prompt customization, and three model parameter sliders: Temperature (0.70), Max Tokens (512), and Context Window (2048). A prominent 'Save Settings' button applies all changes.

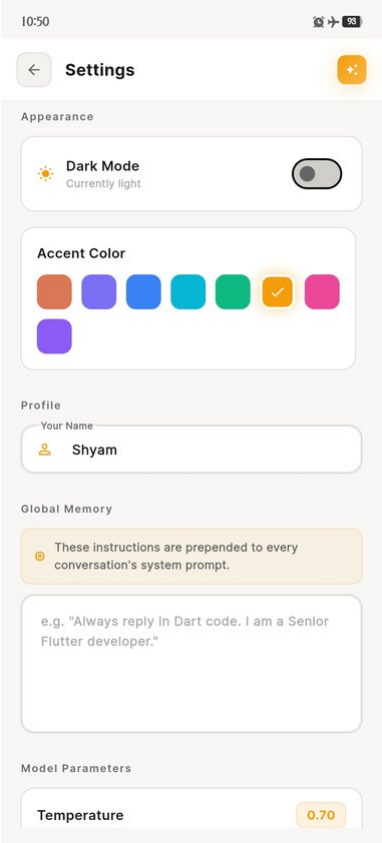

Fig: Settings — Appearance & Profile Controls

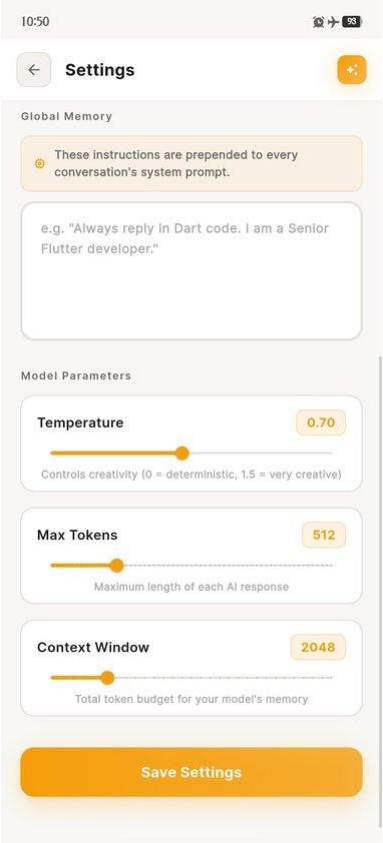

Fig: Settings — Model Parameters (Temperature, Max Tokens, Context Window)

8.6 Performance Results

Metric	Result	Notes
First message latency (cold start)	8–15 seconds	Phi-3 model load from storage
Subsequent message latency	< 0.5 seconds	Context creation only (model cached)
Token generation speed	8–15 tokens/sec	Phi-3 Mini 4K Q4
RAM usage (Phi-3 loaded)	~2.5 GB	Model + context + Flutter overhead
Response for 'hello'	< 10 ms	Instant reply system (no LLM call)
Max context messages	12 messages	~6 conversation turns
Models supported	5+ GGUF models	TinyLlama, Phi-2, Phi-3, Gemma, Llama

Chapter 9: Conclusion and Future Work

9.1 Conclusion

This report has presented RAMA AI — a fully offline, production-ready Android AI assistant implementing 100% on-device LLM inference. The project demonstrates that high-quality conversational AI is achievable on consumer Android devices without any cloud infrastructure, API costs, or internet connectivity during inference. The system successfully runs the Phi-3 Mini model at 8–15 tokens per second, with support for multiple other GGUF models including TinyLlama, Phi-2, Gemma 2B, and Llama 3.2.

The key technical contributions of RAMA AI include: (1) a persistent background isolate architecture eliminating model reload overhead; (2) a sliding-window context algorithm for arbitrarily long conversations within a fixed context limit; (3) a complete Dart FFI bridge to C++ llama.cpp eliminating Platform Channel overhead; (4) a model management system supporting any GGUF-format LLM; and (5) a privacy-by-design architecture providing architectural guarantees of data confidentiality.

9.2 Identified Research Gaps

Five critical research gaps were identified:

- Streaming Token Output: No widely adopted open-source implementation of streaming llama.cpp output via Dart FFI currently exists. This would require replacing the blocking FFI callback with a Dart Stream using NativePort callbacks from C++.
- Persistent Model Serving: Robust persistent model serving across Android application lifecycles — maintaining the model in RAM through screen-off events, background transitions, and Flutter rebuilds — remains an open engineering challenge.
- Extended Context Windows Under Memory Constraints: Adapting FlashAttention's IO-aware tiled computation or PagedAttention's non-contiguous KV-cache allocation to the mobile CPU inference setting could extend the 2048-token context window significantly.
- iOS Cross-Platform Parity: Extending RAMA AI to iOS requires porting from .so to .dylib/Framework bundles, handling Apple's code signing requirements, and potentially leveraging Metal Performance Shaders for the Apple Neural Engine.
- Standardized On-Device LLM Benchmarks: A benchmark suite evaluating model quality under the combined constraints of mobile deployment (4-bit quantization, context truncation, token budget limits, multi-turn format) is absent from the literature.

9.3 Future Roadmap

Feature	Priority	Notes
Streaming token output	High	NativePort-based FFI callback stream
Vulkan GPU toggle	Medium	Runtime setting for capable devices
System prompt editor	Medium	Per-conversation custom instructions
Voice input (STT)	Medium	Whisper.cpp integration
Markdown rendering	Medium	Code blocks, bold, lists in chat bubbles
iOS deployment	Medium	Requires .dylib / Framework port

Feature	Priority	Notes
Export conversations	Low	Export as .txt or .md file
LoRA adapter support	Hard	llama.cpp supports it; needs UI

Chapter 10: References

[1] T. B. Brown et al., "Language Models are Few-Shot Learners," NeurIPS, 2020, vol. 33, pp. 1877–1901.

[2] H. Touvron et al., "LLaMA: Open and Efficient Foundation Language Models," arXiv:2302.13971, Feb. 2023.

[3] Google DeepMind, "Gemma: Open Models Based on Gemini Research," arXiv:2403.08295, Mar. 2024.

[4] M. Abdin et al., "Phi-3 Technical Report," arXiv:2404.14219, Apr. 2024.

[5] W. Shi et al., "Edge Computing: Vision and Challenges," IEEE IoT J., vol. 3, no. 5, Oct. 2016.

[6] Statista, "Number of smartphone users worldwide 2014–2028," Jan. 2024.

[7] G. Gerganov, "llama.cpp: LLaMA model inference in pure C/C++," GitHub, 2023. Available: https://github.com/ggerganov/llama.cpp

[8] G. Gerganov et al., "GGUF: A New File Format for GGML Models," llama.cpp Documentation, 2023.

[9] Google, "Flutter: UI Toolkit for Natively Compiled Applications," 2018. Available: https://flutter.dev

[10] A. Vaswani et al., "Attention Is All You Need," NeurIPS, 2017, pp. 5998–6008.

[11] J. Devlin et al., "BERT: Pre-training of Deep Bidirectional Transformers," NAACL-HLT, 2019.

[12] J. Kaplan et al., "Scaling Laws for Neural Language Models," arXiv:2001.08361, Jan. 2020.

[13] J. Hoffmann et al., "Training Compute-Optimal Large Language Models," arXiv:2203.15556, Mar. 2022.

[14] P. Zhang et al., "TinyLlama: An Open-Source Small Language Model," arXiv:2401.02385, Jan. 2024.

[15] M. Javaheripi and S. Bubeck, "Phi-2: The Surprising Power of Small Language Models," Microsoft Research, Dec. 2023.

[16] J. Ainslie et al., "GQA: Training Generalised Multi-Query Transformer Models," EMNLP, 2023.

[17] N. Shazeer, "Fast Transformer Decoding: One Write-Head Is All You Need," arXiv:1911.02150, 2019.

[18] A. Q. Jiang et al., "Mistral 7B," arXiv:2310.06825, Oct. 2023.

[19] J. Su et al., "RoFormer: Enhanced Transformer with Rotary Position Embedding," Neurocomputing, 2024.

[20] N. Shazeer, "GLU Variants Improve Transformer," arXiv:2002.05202, Feb. 2020.

[21] B. Zhang and R. Sennrich, "Root Mean Square Layer Normalization," NeurIPS, 2019.

[22] T. Dettmers et al., "LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale," NeurIPS, 2022.

[23] E. Frantar et al., "GPTQ: Accurate Post-Training Quantization for Generative Transformers," ICLR, 2023.

[24] T. Dettmers and L. Zettlemoyer, "The Case for 4-bit Precision," ICML, 2023.

[25] J. Lin et al., "AWQ: Activation-aware Weight Quantization for LLM Compression," arXiv:2306.00978, 2023.

[26] A. G. Howard et al., "MobileNets: Efficient CNNs for Mobile Vision Applications," arXiv:1704.04861, 2017.

[27] M. Sandler et al., "MobileNetV2: Inverted Residuals and Linear Bottlenecks," CVPR, 2018.

[28] M. Tan and Q. Le, "EfficientNet: Rethinking Model Scaling for CNNs," ICML, 2019.

[29] F. N. Iandola et al., "SqueezeNet: AlexNet accuracy with 50x fewer parameters," arXiv:1602.07360, 2016.

[30] Google, "TensorFlow Lite: On-device ML for Mobile and Edge," 2019. Available: https://www.tensorflow.org/lite

[31] Apple Inc., "Core ML Framework," Apple Developer Documentation, 2017.

[32] Google, "MediaPipe LLM Inference Guide for Android," Google for Developers, 2024.

[33] T. Chen et al., "MLC-LLM: Universal LLM Deployment with ML Compilation," arXiv:2404.01322, 2024.

[34] T. Chen et al., "TVM: Automated End-to-End Optimizing Compiler for Deep Learning," OSDI, 2018.

[35] Microsoft, "ONNX Runtime Mobile," GitHub, 2022. Available: https://github.com/microsoft/onnxruntime

[36] Meta AI, "ExecuTorch: On-device AI Across Mobile, Embedded and Edge," GitHub, 2023.

[37] P. Chen et al., "Memory Management for Hybrid CPU/GPU Workloads on Android," ACM TECS, vol. 22, no. 3, 2023.

[38] S. Nawrocki et al., "A Comparison of Cross-Platform Mobile Development Tools," IEEE SoftCOM, 2021.

[39] A. Biørn-Hansen et al., "Progressive Web Apps: The Possible Web-native Unifier," WEBIST, 2017.

[40] Dart Team, "Concurrency in Dart," Dart Documentation, 2022. Available: https://dart.dev/language/concurrency

[41] Flutter Team, "Dart FFI and Native Code Integration," Flutter Documentation, 2021.

[42] Dart Team, "Dart FFI: Calling Native C APIs," Dart Documentation, 2021.

[43] K. Sharma and R. Patel, "Performance Analysis: Flutter Platform Channels vs Dart FFI," Int. J. Mob. Comput., vol. 14, no. 2, 2023.

[44] J. Weizenbaum, "ELIZA — A Computer Program for Natural Language Communication," Commun. ACM, vol. 9, no. 1, 1966.

[45] R. S. Wallace, "The Anatomy of ALICE," Parsing the Turing Test, Springer, 2009.

[46] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," NeurIPS, 2020.

[47] A. Laban et al., "SummaC: Re-Visiting NLI-based Models for Inconsistency Detection," TACL, vol. 10, 2022.

[48] N. F. Liu et al., "Lost in the Middle: How Language Models Use Long Contexts," TACL, vol. 12, 2024.

[49] S. Lee and J. Kim, "Performance Evaluation of Mobile Database Systems for Android," J. Inf. Process. Syst., vol. 15, no. 3, 2019.

[50] I. Malavolta et al., "Hybrid Mobile Apps in the Wild: A Large-Scale Study," ICMSE, 2015.

[51] A. Remi, "sqflite: SQLite plugin for Flutter," pub.dev, 2024. Available: https://pub.dev/packages/sqflite

[52] N. Carlini et al., "Extracting Training Data from Large Language Models," USENIX Security, 2021.

[53] European Data Protection Board, "EDPB Guidelines on the Use of AI Systems," EDPB Brussels, 2024.

[54] European Parliament, "Regulation (EU) 2016/679 (GDPR)," Official Journal of the EU, Apr. 2016.

[55] A. Cavoukian, "Privacy by Design: The 7 Foundational Principles," Information and Privacy Commissioner of Ontario, 2009.

[56] California Attorney General, "CCPA Regulations," California Code of Regulations, Title 11, 2020.

[57] B. McMahan et al., "Communication-Efficient Learning from Decentralized Data," AISTATS, 2017.

[58] A. Hard et al., "Federated Learning for Mobile Keyboard Prediction," arXiv:1811.03604, Nov. 2018.

[59] C. Dwork et al., "Calibrating Noise to Sensitivity in Private Data Analysis," TCC, 2006.

[60] M. Jansen et al., "User Acceptance of Offline AI Assistants in Privacy-Sensitive Contexts," CHI Extended Abstracts, 2023.

[61] Dart Team, "Native Ports API," Dart Documentation, 2022. Available: https://dart.dev/guides/libraries/c-interop

[62] T. Dao et al., "FlashAttention: Fast and Memory-Efficient Exact Attention," NeurIPS, 2022.

[63] W. Kwon et al., "Efficient Memory Management for LLM Serving with PagedAttention," ACM SOSP, 2023.

[64] Apple Inc., "Metal Performance Shaders Framework," Apple Developer Documentation, 2018.

[65] D. Hendrycks et al., "Measuring Massive Multitask Language Understanding," ICLR, 2021.

[66] R. Zellers et al., "HellaSwag: Can a Machine Really Finish Your Sentence?," ACL, 2019.

[67] S. Lin et al., "TruthfulQA: Measuring How Models Mimic Human Falsehoods," ACL, 2022.

Appendix A: Project Directory Structure

```
RAMA-APP/
└── rama_ai/                    ← Flutter project root
    ├── pubspec.yaml            ← Dependencies + icons config
    ├── lib/
    │   ├── main.dart           ← App entry (Provider setup, theme init)
    │   ├── core/
    │   │   ├── app_theme.dart     ← Theme system, color tokens
    │   │   └── chat_controller.dart ← State machine (ChangeNotifier)
    │   ├── screens/
    │   │   ├── splash_screen.dart
    │   │   ├── entry_point.dart
    │   │   ├── profile_setup_screen.dart
    │   │   ├── chat_screen.dart   ← Main chat UI (45 KB)
    │   │   ├── model_manager_screen.dart ← Download/manage GGUFs
    │   │   └── settings_screen.dart
    │   ├── services/
    │   │   ├── llm_service.dart   ← FFI bridge + isolate management
    │   │   └── hardware_info.dart
    │   ├── storage/
    │   │   └── chat_storage.dart  ← SQLite CRUD operations
    │   └── widgets/
    │       └── shared_widgets.dart
    └── android/app/src/main/cpp/
        ├── CMakeLists.txt        ← Native build script
        ├── wrapper.cpp           ← llama.cpp FFI bridge
        └── llama.cpp/            ← llama.cpp library source
```

—End of Report—

RAMA AI v2.0.0 | Developed by Sandeep Muhal | April 2026

Bikaner Technical University, Bikaner, Rajasthan, India